2023 Twelfth International Conference on Image Processing Theory, Tools and Applications (IPTA) | 979-8-3503-2541-6/23/$31.00 ©2023 IEEE | DOI: 10.1109/IPTA59101.2023.10320060

Visualization for Multivariate Gaussian Anomaly Detection in Images

João P C Bertoldo*
Centre for Mathematical Morphology (CMM)
Mines Paris - PSL University
Fontainebleau, France
jpcbertoldo@minesparis.psl.eu

David Arrustico*
Centre for Mathematical Morphology (CMM)
Mines Paris - PSL University
Fontainebleau, France
david-sergio.arrustico_villanueva@etu.minesparis.psl.eu

***Abstract*—This paper introduces a simplified variation of the PaDiM (Pixel-Wise Anomaly Detection through Instance Modeling) method for anomaly detection in images, fitting a single multivariate Gaussian (MVG) distribution to the feature vectors extracted from a backbone convolutional neural network (CNN) and using their Mahalanobis distance as the anomaly score. We introduce an intermediate step in this framework by applying a whitening transformation to the feature vectors, which enables the generation of heatmaps capable of visually explaining the features learned by the MVG. The proposed technique is evaluated on the MVTec-AD dataset, and the results show the importance of visual model validation, providing insights into issues in this framework that were otherwise invisible. The visualizations generated for this paper are publicly available at https://doi.org/10.5281/zenodo.7937978.**

***Index Terms*—Anomaly detection, Multivariate Gaussian, Visualization, Whitening, Mahalanobis distance**

I. Introduction

Anomaly detection (AD) consists of identifying samples that deviate from a semantic concept [1]. It is important in many fields because it allows one to flag unexpected data points, which may represent errors, malfunctions, fraud, etc. By detecting anomalies, one can identify and mitigate potential problems before they cause significant damage, and gain insights into the underlying data that might not be apparent otherwise. In image AD, these findings can be done at different scales, most commonly at the pixel and image levels.

In recent years, deep learning techniques for AD tasks have gained considerable attention [2], which can be attributed to their ability to generate meaningful data representations in the form of feature maps. In this line, ResNet [3] marked a notable breakthrough in the field of deep neural networks, for instance, by achieving success on ImageNet tasks thanks to the use of residual representations and shortcut connections.

In response to the growing demand for publicly available image datasets for AD research, [4] introduced the MVTec Anomaly Detection (MVTec-AD) dataset, which comprises over 5000 images across 15 categories. Anomalies in this dataset are understood as defects on textures and objects (i.e. "normal" images consist of defectless samples), and their masks (pixel-wise labels) are provided at high resolution.

Previous works have exploited the use of multivariate Gaussian (MVG) distributions and the Mahalanobis distance (M. distance) to detect anomalies in MVTec-AD in an unsupervised way. The framework proposed in [5] first attacked the problem at the image level and posterior works such as [6] have adapted their approach to the pixel level. Despite their reasonable performances, these papers lack visual explanations capable of validating or debugging their models, which we aim to cover in this paper.

In a related vein, the field of neural network visualization has also witnessed significant breakthroughs. For instance, [7] successfully identified units within image classifiers that corresponded to human-interpretable concepts. Inspired by their visualization strategy, the present paper aims to adapt their approach to Gaussian AD models.

II. Method

We propose a simplified variation of Patch Distribution Modeling Framework (PaDiM) [6] which, instead of fitting one Gaussian distribution per pixel position, fits all pixel-wise feature vectors under a single distribution.

A. Feature maps

Let $x \in \mathbb{R}^{H \times W}$ be a feature map extracted from a backbone convolutional neural network (CNN), where H and W are, respectively, its height and width – notice that they are generally smaller than the image's height and width. The collection of feature maps (stack operation) from a single layer of the backbone CNN is denoted $X \in \mathbb{R}^{C \times H \times W}$, where C is its number of channels (i.e. different feature maps). A collection of feature maps from B images is denoted, in bold, $\boldsymbol{X} \in \mathbb{R}^{B \times C \times H \times W}$, and we add subscripts to differentiate the train set ($\boldsymbol{X}_{\text{train}}$; only normal images) and the test set ($\boldsymbol{X}_{\text{test}}$; normal and anomalous images).

We use the term "[pixel-wise] feature vector" and denote it as $X_{:hw}$ to refer to the vector along the channel dimension ($:$ denotes "all values") at height position $h \in [\![1, H]\!]$ and width position $w \in [\![1, W]\!]$ of the feature maps X, where $[\![1, I]\!]$ denotes the set of integer values $\{1, \dots, I\}$.

*These authors equally contributed to this work.

979-8-3503-2541-6/23/$31.00 ©2023 IEEE

Authorized licensed use limited to: Universita degli Studi di Roma La Sapienza. Downloaded on May 06,2026 at 07:43:19 UTC from IEEE Xplore. Restrictions apply.

B. Whitened feature maps

We fit a multivariate Gaussian (MVG) distribution with the feature vectors in $\boldsymbol{X}_{\text{train}}$ with all pixel positions confounded. A flattening operation is done on $\boldsymbol{X}_{\text{train}}$ so that all dimensions except the channel dimension are reduced. Thus, we obtain $\boldsymbol{X}_{\text{train}}^{\textit{flat}} \in \mathbb{R}^{BHW \times C}$, from which the empirical mean vector $\hat{\mu} \in \mathbb{R}^C$ and the unbiased empirical covariance matrix $\hat{\Sigma} \in \mathbb{R}^{C \times C}$ are computed, respectively, by

$$\hat{\mu} = \frac{1}{BHW} \sum_{i=1}^{BHW} \left(\boldsymbol{X}_{\text{train}}^{\textit{flat}} \right)_{i:} \tag{1}$$

and

$$\hat{\Sigma} = \frac{1}{BHW - 1} \sum_{i=1}^{BHW} \left[\left(\boldsymbol{X}_{\text{train}}^{\textit{flat}} \right)_{i:} - \hat{\mu} \right] \left[\left(\boldsymbol{X}_{\text{train}}^{\textit{flat}} \right)_{i:} - \hat{\mu} \right]^T . \tag{2}$$

At inference time, the Mahalanobis distance (M. distance) of a feature vector $X_{:hw}$

$$M\left(X_{:hw}\right) = \sqrt{\left(X_{:hw} - \hat{\mu}\right)^T \hat{\Sigma}^{-1} \left(X_{:hw} - \hat{\mu}\right)} \tag{3}$$

is assigned to each pixel in the feature maps resolution as its anomaly score (higher means "more anomalous"). However, Eq. 3 can be rewritten as

$$\begin{aligned} M^2\left(X_{:hw}\right) &= \left(X_{:hw} - \hat{\mu}\right)^T \hat{\Sigma}^{-1} \left(X_{:hw} - \hat{\mu}\right) \\ &= \left(X_{:hw} - \hat{\mu}\right)^T \left(\hat{\Sigma}^{-1}\right)^{\frac{1}{2}} \left(\hat{\Sigma}^{-1}\right)^{\frac{1}{2}} \left(X_{:hw} - \hat{\mu}\right) \\ &= \left[\left(\hat{\Sigma}^{-1}\right)^{\frac{1}{2}} \left(X_{:hw} - \hat{\mu}\right)\right]^T \left[\left(\hat{\Sigma}^{-1}\right)^{\frac{1}{2}} \left(X_{:hw} - \hat{\mu}\right)\right] \\ &= \left[Y_{:hw}\right]^T \; Y_{:hw} = \| \, Y_{:hw} \, \|_2^2 \, , \end{aligned} \tag{4}$$

where $\left(\hat{\Sigma}^{-1}\right)^{\frac{1}{2}}$ is the square root matrix of $\hat{\Sigma}^{-1}$ (*not* the point-wise square root operator) and

$$Y_{:hw} = \left(\hat{\Sigma}^{-1}\right)^{\frac{1}{2}} \left(X_{:hw} - \hat{\mu}\right) \tag{5}$$

is the result of an empirical whitening transformation[1] applied to $X_{:hw}$, therefore we name it a *whitened* feature vector. Analogously, $Y \in \mathbb{R}^{C \times H \times W}$ is referred to as the *whitened* feature maps, and its channels are referred to as "components" as a reminder that they correspond to the projection of the (centered) feature vector on each *eigencomponent* of $\hat{\Sigma}^{-1}$.

[1]Eq. 5 is called "whitening" transformation because the covariance matrix of its image on the training data is the identity matrix, therefore a white noise. "Empirical" refers to using the empirical parameters $\hat{\mu}$ and $\hat{\Sigma}$.

C. Squared whitened feature maps

As shown in Eq. 4, the M. distance of $X_{:hw}$ is equivalent to Euclidean norm of $Y_{:hw}$. Since $\cdot^2$ is monotonic, the squared M. distance $M^2\left(\cdot\right)$ has the same ordering than $M\left(\cdot\right)$. Thus the anomaly score of $X_{:hw}$ can be explained as the sum of the components of the *squared* whitened feature vector $\left(Y^2\right)_{:hw}^2$, where $\cdot^2$ is a point-wise operator, as

$$\begin{aligned} M^2\left(X_{:hw}\right) &= \| \, Y_{:hw} \, \|_2^2 \\ &= \| \left(Y^2\right)_{:hw} \|_1 = \sum_{c=1}^{C} \left(Y^2\right)_{chw} \, . \end{aligned} \tag{6}$$

III. Experiments

We leverage the equivalence proposed in Eq. 6 to generate visual explanations of the anomaly score maps $M^2\left(X\right) \in \left(\mathbb{R}_+\right)^{H \times W}$, where

$$\left(M^2\left(X\right)\right)_{hw} = M^2\left(X_{:hw}\right) \; \forall \left(h, w\right) \in [\![1, H]\!] \times [\![1, W]\!] \, . \tag{7}$$

A. Backbone

We use a ResNet-18 [3] pre-trained on the ImageNet dataset (`torchvision` version `0.15.2`, weights `ResNet18_Weights.IMAGENET1K_V1`) as the backbone CNN to extract feature maps. The feature maps are extracted from its four major blocks: "layer1", "layer2", "layer3", and "layer4". The figures with their visualizations (Sec. IV) indicate their anomaly segmentation performances on the test set in terms of pixel-wise Area Under the Receiver Operating Characteristic curve (AUROC), pixel-wise Area Under the Precision-Recall curve (AUPR), and Area Under the Per-Region Overlap curve (AUPRO) [4].

B. Data

The model proposed in Eq. 1 and Eq. 2 is independently trained on each layer of the backbone CNN and each category of the MVTec-AD dataset [4]. Then, the proposed whitening transformation (Eq. 5) is applied on the feature maps. Finally, we generate the *squared* whitened feature maps Y^2 (point-wise square of Y).

All the visualizations are publicly available on Zenodo at https://doi.org/10.5281/zenodo.7937978 [8] However, this paper is limited to the category "hazelnut" for the sake of space and simplicity. Similar observations were seen in other categories.

C. Visualizations

All visualizations consist of a superposition of two images using alpha blending with $\alpha = 0.5$: the input image and a *heatmap*, which can be a *squared component map* $y^2 = \left(Y^2\right)_{c::}$ of a given component c or the squared anomaly score map $M^2\left(X\right)$. The heatmaps are upsampled to 224×224 to match the input image resolution[2]. We use OpenCV (Python

[2]$M^2\left(X\right)$ is first computed in its own (lower) resolution and then resized.

Authorized licensed use limited to: Universita degli Studi di Roma La Sapienza. Downloaded on May 06,2026 at 07:43:19 UTC from IEEE Xplore. Restrictions apply.

library version `4.7.0.68`) with bilinear interpolation and the plots are generated with `matplotlib` (version `3.6.3`).

D. Color scale

The heatmaps use the color scale "inferno" with the minimum value set to zero and the maximum value set to a per-plot statistic. In simple terms, we use percentile values to avoid losing color resolution when extreme values cause the maximum to explode. For the heatmaps of $M^2(X)$, the maximum value is set to the 99th percentile of all values of $M^2(X)$ in the dataset split (train or test) being visualized. For the heatmaps of y^2, we propose two color scaling strategies, which serve different purposes, referred to as "per-component color scale" and "cross-component color scale".

Per-component color scale: a different maximum value is chosen for each component independently; the color scale's maximum value of a fixed component c is the 99th percentile of all values of $\left(\boldsymbol{Y}^2\right)_{:c::}$ in the dataset split (train or test) being visualized. This strategy is useful to understand the information encoded in each component because it provides better visibility of the nuances across different images.

Cross-component color scale: a single maximum value is chosen for all components; the color scale's maximum value is the 99th percentile of all values of $\boldsymbol{Y}^2$ in the dataset split (train or test) being visualized. With this strategy, all the heatmaps are comparable in magnitude, so the "visual sum" of the heatmaps from the same image corresponds to the anomaly score map (their component-wise sum, cf. Eq. 6).

E. Component Sorting (Single-component AUROC)

As a result of the whitening transformation[3], the value of Y_{chw} corresponds to the projection of the vector $X_{:hw} - \hat{\mu}$ onto the eigenvector (from $\hat{\Sigma}$) with the c-th smallest eigenvalue associated to it. In other words, the channels of Y are sorted by the eigenvalues of $\hat{\Sigma}$ in ascending order. However, we found it more helpful to sort them by their individual performances. Each squared component y^2 is interpreted as anomaly score map (as if Y was reduced to a single component) and its respective (pixel-wise) AUROC, later referred to as "single-component AUROC", is used to sort the components of each layer separately.

IV. Results and Discussion

Visual inspection of a large number of images revealed that some weaknesses of the model can be easily identified, but understanding how components interact and build a correct anomaly score map is not as simple. We present a selection of visualizations that summarize the major insights learned.

The upper title of each figure displays the segmentation performance of the model being visualized (using all the components). Moreover, on top of each visualization, the index of the image and the index of the component in Y are indicated. "In-plot" indices refer to the (1-starting) index in the grid of that figure. "Index" indices refer to the (0-starting) index of the image as sorted in MVTec-AD, and to the (0-starting) index of the component as sorted by increasing order of eigenvalues from $\hat{\Sigma}$ (cf. Sec. III-E).

First, we focus our analysis on *bad* (low single-component AUROC) components, whose weaknesses can be seen even on the train set. Then we visualize these same components on test anomalous images, which show that some artifacts are attenuated while others persist. Finally, we show test anomalous images superposed with *good* (high single-component AUROC) components and their final anomaly score map.

A. Low-AUROC components

In this subsection, all heatmaps use the per-component color scale (cf Sec. III-D) to better visualize their artifacts. Fig. 1(a), Fig. 1(b), Fig. 1(c), and Fig. 1(d) display heatmaps of three squared components among the *worst* (lowest single-component AUROC) respectively from layer1, layer2, layer3, and layer4. They all show the same three *normal images from the training set*. Fig. 2(a) and Fig. 2(b) display, respectively, the same layers and components present in Fig. 1(a) and Fig. 1(b), but superposed on *anomalous images from the test set*. Anomalous images from layer3 and layer4 were omitted due to the lack of space but similar phenomena have been observed.

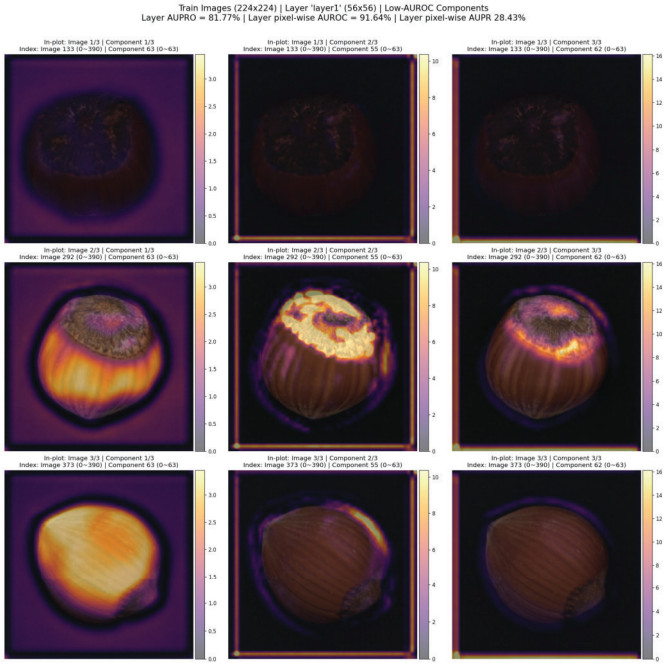
1) Resolution mismatch: [6] proposed that deeper layers achieve lower performances – despite their higher number of features – because they are too biased by the classes from the pre-training dataset, but our results show an alternative (or complementary) explanation. Fig. 1 shows that deeper layers have an increasing problem of resolution mismatch. As downsampling operations are chained, the feature maps' resolutions become too coarse; layer4, for instance, is 7×7 while the input image is 224×224, thus a pixel in the former represents a patch of 32×32 from the latter.

2) Whitened component value ranges: for any component c, 99.7% of the values in $\left(\boldsymbol{Y}_{\text{train}}\right)_{:c::}$ are expected to be in the interval $[-3, 3]$ due to the whitening operation (empirical rule for a standard normal distribution), so the values in $\left(\boldsymbol{Y}^2_{\text{train}}\right)_{:c::}$ are expected to be in the interval [0, 9]. However, when setting the maximum color scale with the 99th percentiles of $\left(\boldsymbol{Y}_{\text{train}}\right)_{:c::}$ (cf. Sec. III-D), inconsistent behavior is observed in some cases. For instance, in Fig. 1, component 62 in layer1 and component 17 in layer4 have saturation values around 16, which is higher than expected, but component 63 in layer1, component 127 in layer2, and component 510 in layer4 have saturation values around $3 \sim 5$, which is lower than expected.

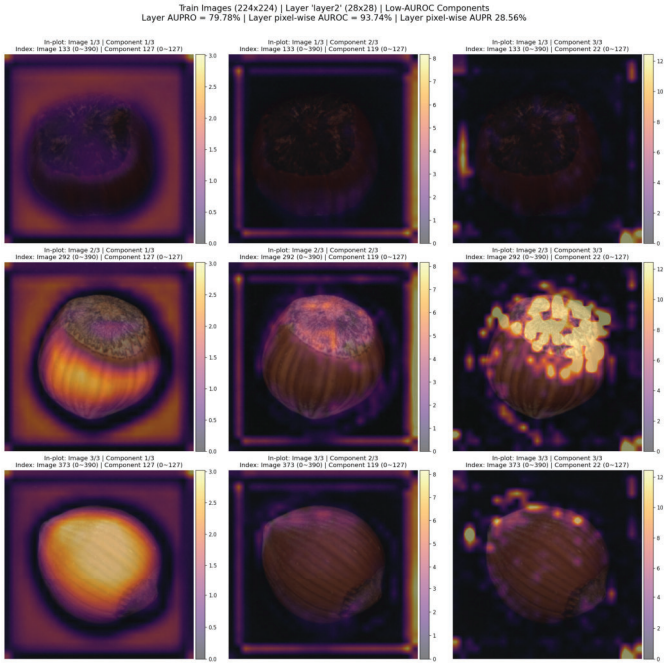
3) Border effects: several components show input-independent artifacts (i.e. they happen no matter the content of the input image) on the borders and corners of the heatmaps. In Fig. 1, components 55 and 62 from layer1, component 119 from layer2, component 133 from layer3, and components 510 and 133 from layer4 show artifacts along the borders of the image (vertical and horizontal strips of high activation). Also in Fig. 1, component 22 from layer2, component 213 from layer3, and component 17 from layer4 similarly show artifacts on the corners of the images. We hypothesize these

[3]We use the method `whiten()` from `scipy.stats.Covariance`, which takes the eigendecomposition of $\hat{\Sigma}$ as constructor arguments.

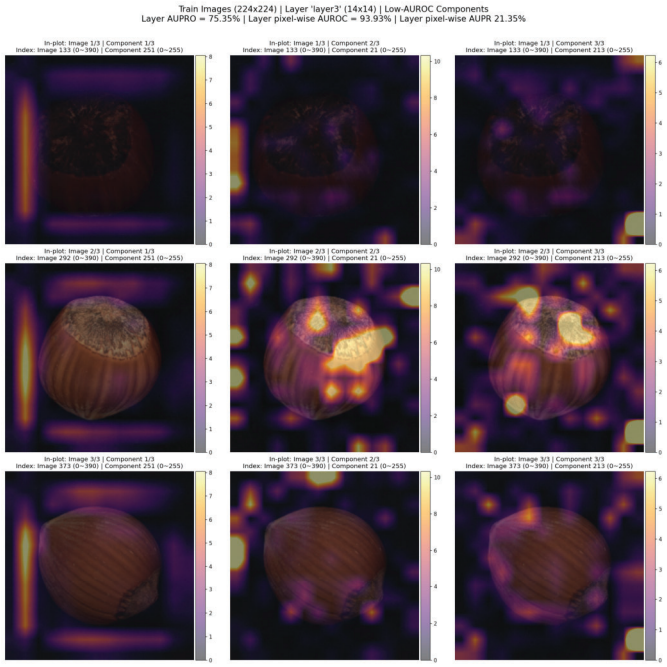
Authorized licensed use limited to: Universita degli Studi di Roma La Sapienza. Downloaded on May 06,2026 at 07:43:19 UTC from IEEE Xplore. Restrictions apply.



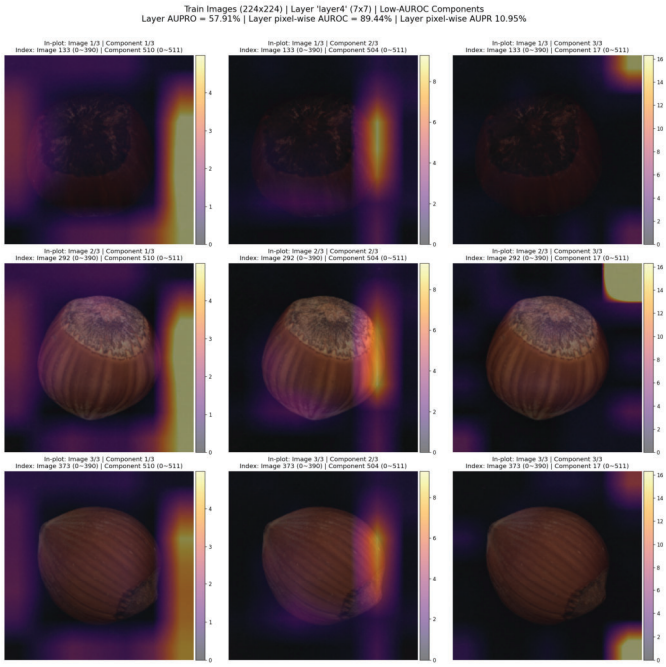
(a) layer1



(b) layer2



(c) layer3



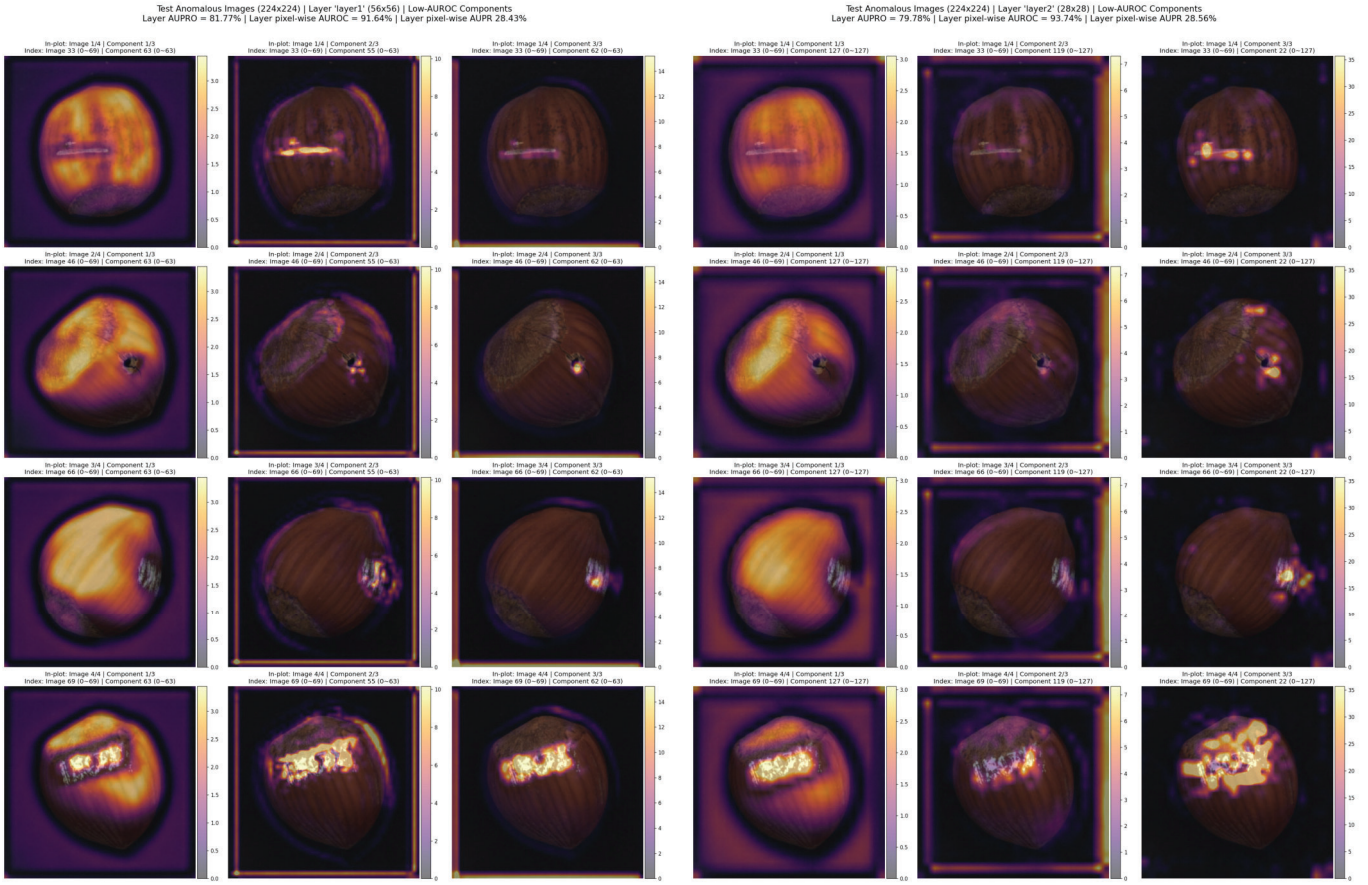
(d) layer4

Fig. 1: Visualizations on training (normal) images. Selection of low-AUROC components.

issues ensue from the padding used in the backbone CNN: as multiple padded convolutions are successively applied, the border effects accumulate and the feature vectors near the borders shift to a different distribution mode, which makes them unusual (higher anomaly score) compared to the pixels inside the image, which are the majority.

4) Some border effects are worse than others: the visualizations of anomalous images in Fig. 2 show that the presence of border effects does not necessarily indicate that a component is bad for detecting anomalous pixels. For component 33 from layer1 and component 119 from layer2, the border effects are so pronounced that even real anomalies have lower values (cf. images 33, 46, and 66). However, component 22 in layer2 activates more intensely on the anomalies than on the corner artifacts, making them practically disappear; notice that the upper bound of the color scale in Fig. 2(b) is higher than in

Authorized licensed use limited to: Universita degli Studi di Roma La Sapienza. Downloaded on May 06,2026 at 07:43:19 UTC from IEEE Xplore. Restrictions apply.



(a) layer1 (b) layer2

Fig. 2: Visualizations on test anomalous images. Selection of low-AUROC components.

Fig. 1(b) (≈ 35 against ≈ 12 respectively).

5) Foreground-background frontier: particularly within object categories (e.g. "hazelnut"), several components seem to detect the border between the object (foreground) and the background. When this behavior is observed, in most cases, the foreground-background border shows high activation. Interestingly, the opposite also happens in a few cases such as in Fig. 1 for component 63 in layer1 and component 127 in layer2. Notice that this phenomenon persists in anomalous images (Fig. 2), and some anomalies even have low values compared to the normal foreground and background (e.g. images 46 and 66 in Fig. 2(a)), which contradicts the expected behavior of the model.

B. High-AUROC components

In this subsection, all heatmaps use the cross-component color scale (cf. Sec. III-D) to better visually compare how the values of different components interact. Fig. 3 displays heatmaps of three components among the *best* (highest single-component AUROC) and the anomaly score map (right-most heatmap at each row) from layer1 and layer2. They both show the same four *anomalous images from the test set* present in Fig. 2. The analogous visualizations from layer3 and layer4 are omitted due to the lack of space but similar phenomena have been observed.

The border/corner artifacts mentioned in Sec. IV-A3 are also present in some cases such as components 13 and 14 in layer1 and components 74 and 75 in layer2, but they all show high values on the *actual* anomalous pixels.

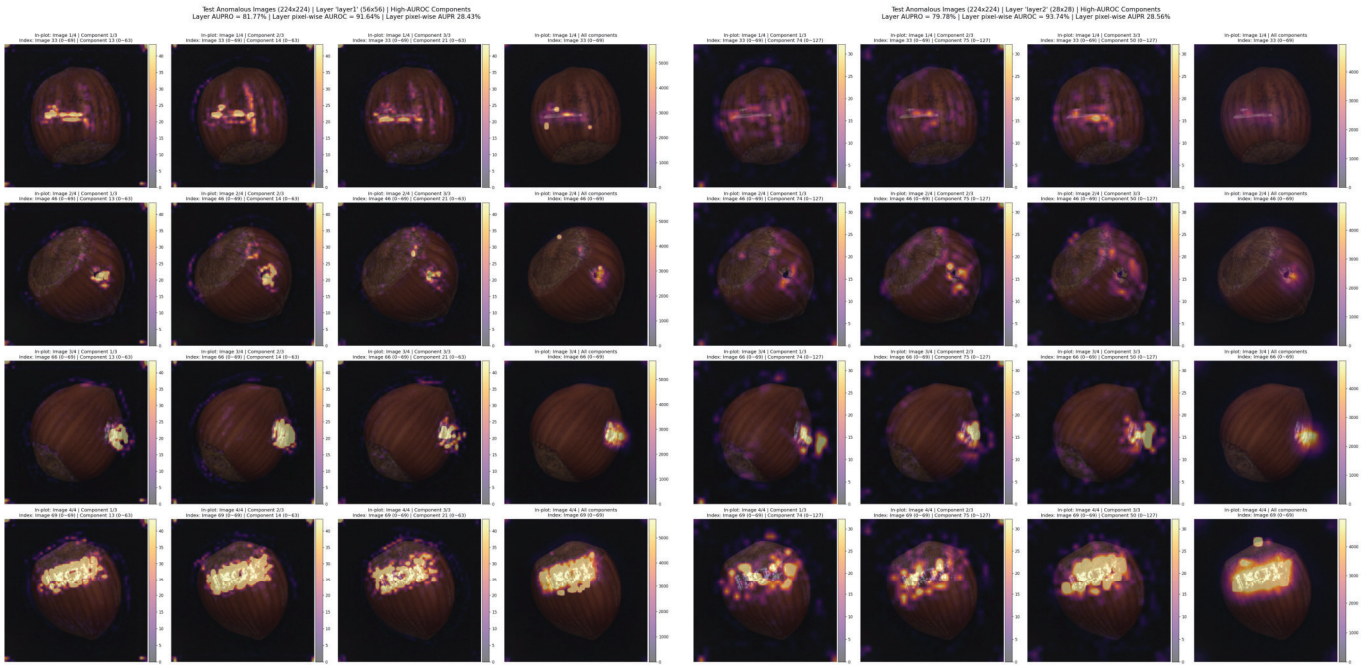
It can also be observed that there is some degree of "agreement" on the anomalous pixels, but rather "disagreement" on the false positive pixels. In other words, the high-value regions on the anomalies (thus true positives) intersect across components, redundantly detecting abnormalities. However, the high-value regions on normal pixels (thus false positives) are randomly placed. The anomaly score map confirms this explanation as the false positive regions are not visible because the sum of true positive values shadows them in terms of magnitude.

Finally, we observed that some types of anomaly have higher scores than others; for instance, compare the anomaly score maps of images 33 and 69 in Fig. 3(b).

V. Conclusion

We proposed a simplified variation of PaDiM [6] for anomaly detection in images that fits a single multivariate Gaussian (MVG) distribution to pixel-wise feature vectors

Authorized licensed use limited to: Universita degli Studi di Roma La Sapienza. Downloaded on May 06,2026 at 07:43:19 UTC from IEEE Xplore. Restrictions apply.



(a) layer1 (b) layer2

Fig. 3: Visualizations on test anomalous images. Selection of high-AUROC components.

extracted from a ResNet-18. We showed that the Mahalanobis distance (M. distance) (i.e. the anomaly score) of each pixel can be expressed as the Euclidean norm of its image from an empirical whitening transformation. Then, we explore this equivalence to generate visual explanations for our model using heatmaps of the individual axes of these *whitened* feature vectors.

The experiments conducted on the MVTec-AD dataset demonstrate the utility of the proposed visualizations to provide insights into the behavior of our model, for instance revealing pixel position-related artifacts and showing that deep feature maps have bad performance because their resolution is too low. Our analysis suggests that the model could be improved by removing artifact-ful heatmaps (i.e. dimension reduction), adjusting the input-feature map resolution mismatch, and taking the padding effects (from the CNN architecture) into account.

REFERENCES

[1] V. Chandola, A. Banerjee, and V. Kumar, "Anomaly detection: A survey," *ACM computing surveys (CSUR)*, vol. 41, no. 3, pp. 1–58, 2009.

[2] L. Ruff, J. R. Kauffmann, R. A. Vandermeulen, *et al.*, "A unifying review of deep and shallow anomaly detection," *Proceedings of the IEEE*, vol. 109, no. 5, pp. 756–795, 2021.

[3] K. He, X. Zhang, S. Ren, and J. Sun, *Deep residual learning for image recognition*, Dec. 10, 2015. DOI: 10.48550/arXiv.1512.03385. arXiv: 1512.03385[cs]. [Online]. Available: http://arxiv.org/abs/1512.03385 (visited on 05/26/2023).

[4] P. Bergmann, M. Fauser, D. Sattlegger, and C. Steger, "MVTec AD – a comprehensive real-world dataset for unsupervised anomaly detection," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, Jun. 2019.

[5] O. Rippel, P. Mertens, E. König, and D. Merhof, "Gaussian anomaly detection by modeling the distribution of normal data in pretrained deep features," *IEEE Transactions on Instrumentation and Measurement*, vol. 70, pp. 1–13, 2021.

[6] T. Defard, A. Setkov, A. Loesch, and R. Audigier, "Padim: A patch distribution modeling framework for anomaly detection and localization," in *Pattern Recognition. ICPR International Workshops and Challenges: Virtual Event, January 10–15, 2021, Proceedings, Part IV*, Springer, 2021, pp. 475–489.

[7] D. Bau, J.-Y. Zhu, H. Strobelt, A. Lapedriza, B. Zhou, and A. Torralba, "Understanding the role of individual units in a deep neural network," *Proceedings of the National Academy of Sciences*, vol. 117, no. 48, pp. 30 071–30 078, 2020.

[8] J. P. C. Bertoldo and D. Arrustico, *Pixel-wise gaussian AD visualization (ResNet18, MVTecAD)*, version 0.0.0, May 15, 2023. DOI: 10.5281/ZENODO.7937978. [Online]. Available: https://zenodo.org/record/7937978 (visited on 05/15/2023).

Authorized licensed use limited to: Universita degli Studi di Roma La Sapienza. Downloaded on May 06,2026 at 07:43:19 UTC from IEEE Xplore. Restrictions apply.